Express.js Audit Report

Project: LWMServer Date: 2025-10-29

Executive Summary

Overall the system is functional and reasonably structured, but several critical security gaps exist around HTTP security headers, lack of input validation and rate limiting, and an exposed Firebase service account in the repository. Performance is acceptable for typical CRUD operations, though some endpoints likely incur heavy database work without caching and there is extensive console logging. Architectural separation is partially applied (controllers, models, routes, middlewares), but there is limited service/repository abstraction, inconsistent validation, and minimal centralized logging/monitoring. Documentation and onboarding are sparse.

Priority remediation items: 1) remove committed secrets, 2) add Helmet, rate limiting, and strict CORS, 3) implement input validation across routes, 4) review CSP to remove unsafe-inline/unsafe-eval, 5) standardize logging and error handling, 6) introduce caching for read-heavy endpoints and pagination defaults.

Detailed Findings

1) Security

Findings:
- No Helmet detected; only a manually set Content-Security-Policy (CSP) is present in server.js with unsafe-inline and unsafe-eval (risk of XSS).

- No express-rate-limit found; sensitive routes (auth, uploads, messaging, Stripe webhook) are unprotected from brute force and abuse.

- CORS is an allowlist but includes localhost and raw IPs; ensure strict env-specific origins.

- No global input validation; many controllers accept body/query directly. utils/APIFeatures enables regex from clients (possible ReDoS) and broad field filtering.

- JWT auth present; access token TTL 30d (long). Refresh token 7d stored on user document; single-device semantics; no device binding.

- Critical: Firebase service account JSON committed and loaded by socket.js; immediate key rotation required.

- Public download route serves files under public/ by name. Path traversal is mitigated by path.join, but consider whitelisting.

- Stripe webhook uses raw body; ensure signature verification in controllers/taskCtrl.handleStripeWebhook.

- Missing HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy beyond CSP.

- Socket.IO authenticates with JWT; CORS credentials enabled; ensure origin list is production-scoped.

Risk Rating:
High: Exposed Firebase key; no Helmet/rate limits; weak CSP.

Medium: Long-lived access tokens; permissive regex queries; dev origins in CORS; public artifacts.

Low: Console logging; inconsistent env names; missing additional headers.

Recommendations:
- Add Helmet with frameguard, noSniff, hsts (prod), referrerPolicy, and CSP without unsafe-inline/eval (use nonces/ CSP hashes for Stripe/PayPal).

- Add express-rate-limit on auth/upload/webhook/messaging; consider slow-down.

- Add validation middleware (Joi/Zod/Celebrate) across all routes; cap regex length and escape patterns.

- Remove committed Firebase JSON; rotate GCP key; load secrets from env/secret manager; add gitleaks.

- Reduce access token TTL to 1560 minutes; rotate refresh tokens per device.

- Add HSTS (behind TLS), X-Content-Type-Options, X-Frame-Options, Referrer-Policy; audit CORS per env.

- Ensure Stripe signature verification and quick 2xx responses with async processing.

2) Performance

Findings:
- Mongoose queries with populate and sometimes no lean(); risk of heavier memory use.
- utils/APIFeatures encourages flexible queries that may miss indexes; regex scans are costly.
- Socket.IO emits frequent live events (typing/recording/location); potential high throughput.
- No caching (HTTP/Redis), no compression, minimal pagination controls on some endpoints.

Recommendations:
- Add indexes for frequent fields (userId, roomId, createdAt, taskStatus, orderNumber). Use .lean() for read-only endpoints.
- Enforce pagination and max limits everywhere; default sensible limits.
- Enable compression and cache headers for static assets; consider CDN for /public.
- Introduce Redis for caching and as Socket.IO adapter if scaling out.
- Offload heavy tasks to background queues (BullMQ).

3) Design & Architecture

Findings:
- MVC-like layout is present, but controllers hold business and data logic; limited services/repositories.
- No dependency injection; config reads sprinkled across files; limited centralized error handling.
- Swagger present but no auth/permission examples per route.

Recommendations:
- Create a service layer; keep controllers thin. Optionally add repositories for data access.
- Add centralized error middleware and structured logger (pino/winston) with request IDs.
- Centralize configuration per env with validation (envalid/joi).
- Wrap external integrations (Stripe/Cloudinary/Firebase) in services for testability.

4) Reusability & Maintainability

Findings:
- Duplicated messaging/notification logic across upload and socket handlers.
- Hardcoded admin ID and scattered origin lists; naming inconsistencies (role vs roleType).
- Extensive console logs; no levels or sinks.

Recommendations:
- Extract shared messaging logic into utilities/services; centralize constants and configuration.
- Standardize naming/linting; add CI checks.
- Introduce unit/integration tests for auth, permissions, messaging, and webhooks.

5) Documentation & Environment

Findings:
- Onboarding docs are sparse; no .env.example. Some focused READMEs exist.

Recommendations:
- Add README with setup, run, test, deploy; add .env.example; add SECURITY.md.

Prioritized Remediation Plan

High: remove committed secrets; add Helmet & rate limit; strict CSP; validation.

Medium: shorten token TTL; add pagination & lean(); centralize logging/config.

Low: cleanup logs; refine CORS; static asset caching/CDN.

Scores (010)

Security: 5.0

Performance: 6.0

Design & Architecture: 6.0

Reusability & Maintainability: 5.5

Documentation & Environment: 3.5